AI CODE REVIEW

The bug wasn't in the diff.

How a code graph gives every reviewer subagent the callers, tests and settings that make a change matter.

deadline.ts · the change

looks fine

```
deadlineFrom(timeoutSec)
```

the bug is hiding here

scheduler.ts · the caller

looks fine

```
deadlineFrom(job.timeoutMs)
```

A helper now expects seconds. Its caller still sends milliseconds.

deadline.ts · just changed

correct

```
function deadlineFrom(timeoutSec) {
  return now() + timeoutSec * 1000
}
```

scheduler.ts · untouched

unchanged

```
const due = deadlineFrom(
  job.timeoutMs    // 30000
)
```

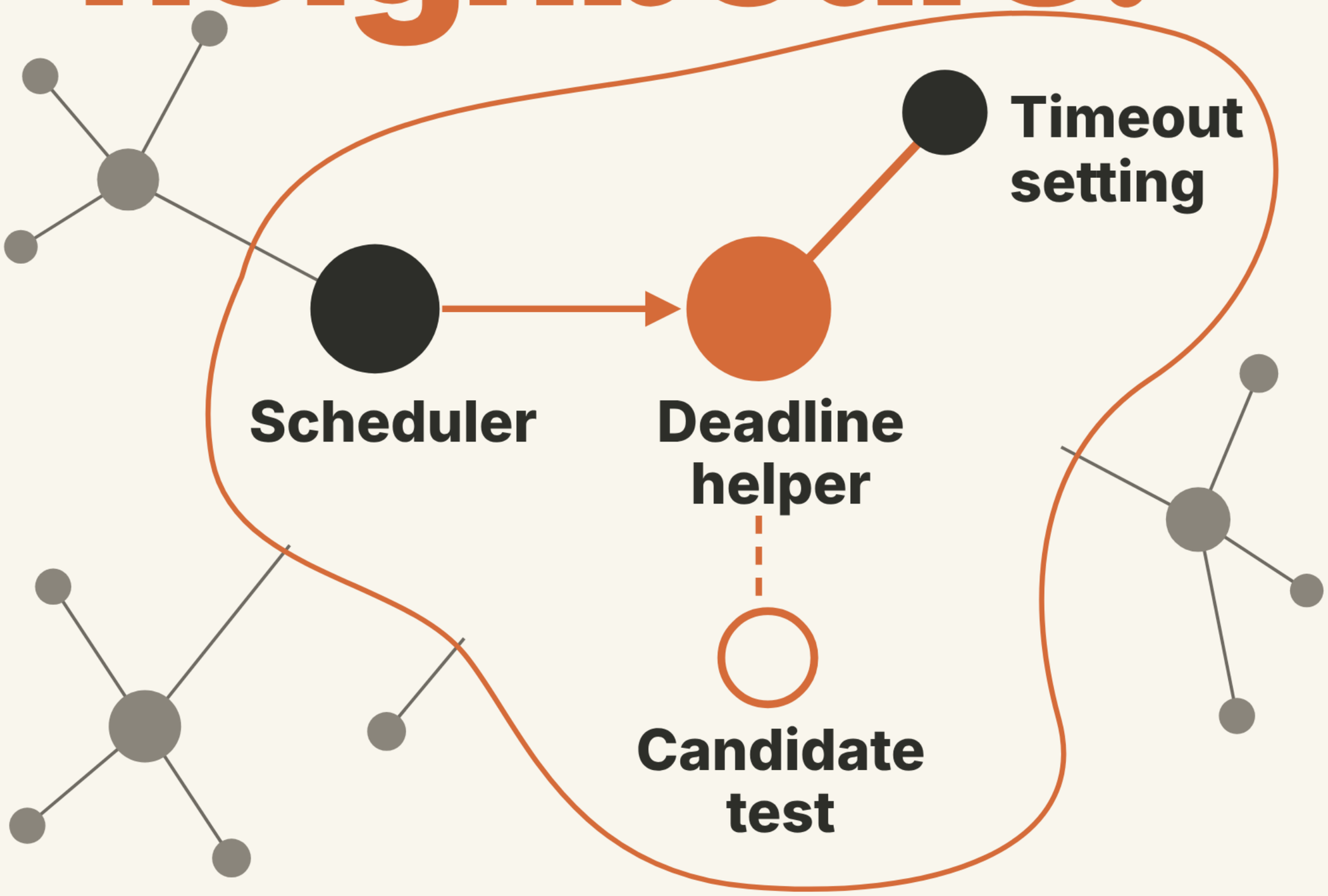
30,000 “seconds” is a deadline 8 hours 20 minutes away.

Read the helper alone and it looks fine.

Put the scheduler’s call beside it and the mistake is obvious.

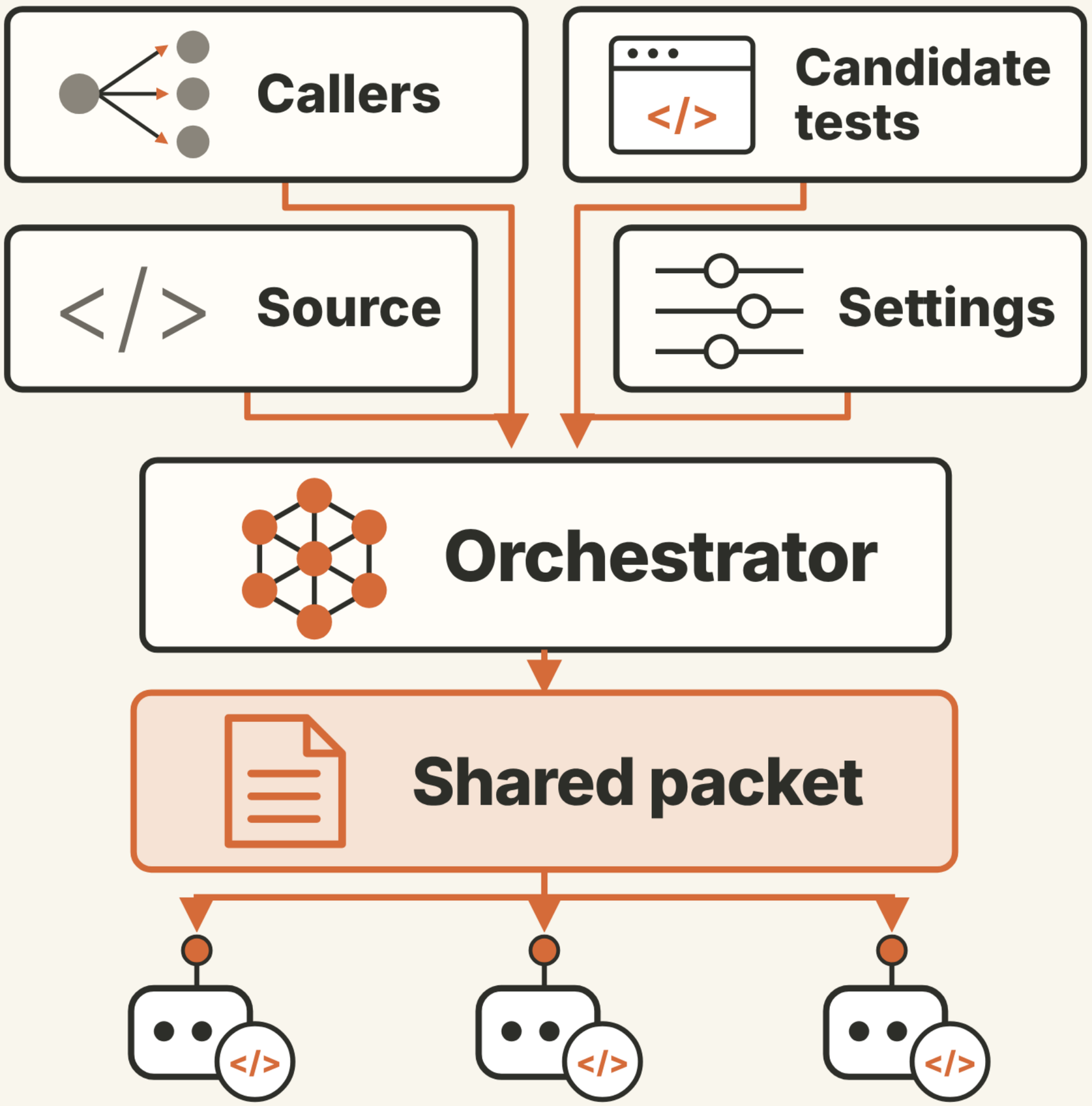
The important fact isn’t in the changed lines. It’s in the connection.

A change has neighbours.



A fresh agent doesn’t know this neighbourhood. So we hand it one.

Build the context once.



The orchestrator packs it once. Every reviewer starts from the same picture.

A shell command, not an MCP server.

MCP server

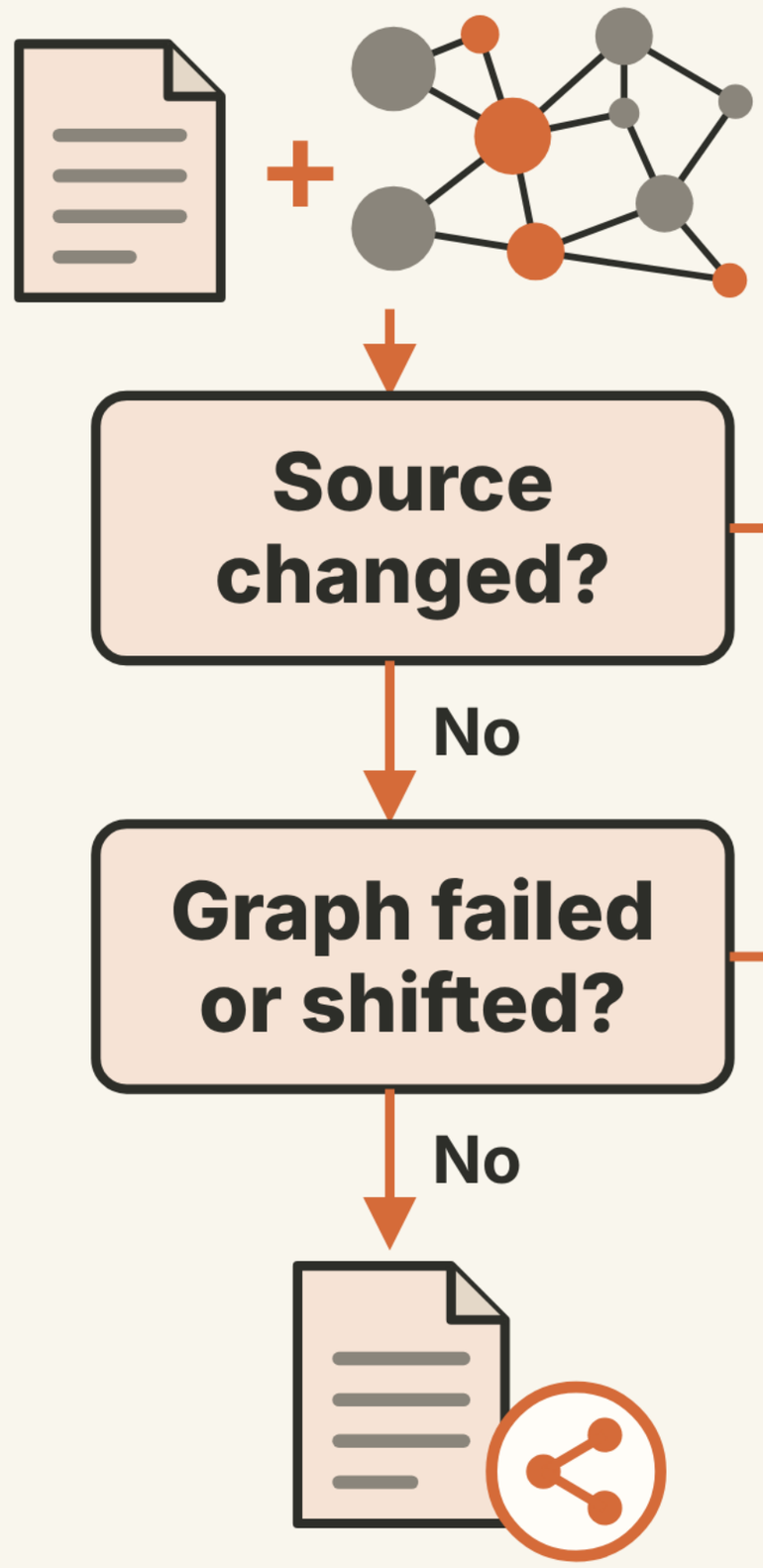
Tool descriptions ride along in context on every turn, even the many that never touch the graph.

```
$ crg
```

Called only when packing needs it. One owner, the orchestrator, keeps the map current.

Don't mix moments.

During collection



Fresh means every piece of evidence comes from the same moment.

Before handing it over.

1 Include the neighbours

Callers, tests and settings beside the change.

2 Check while packing

The map must match the code under review.

3 Share the context

Every reviewer starts from the same evidence.

4 Drop shaky graph results together

Keep the source, say what’s missing.

5 Read the connections

The graph finds evidence. You judge the code.

READ THE FULL POST

A diff shows what changed.

A code graph finds what makes it matter.

The full write-up is on my Substack.
Link in the first comment.

rajsfieldnotes.substack.com